

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



XỬ LÝ NGÔN NGỮ TỰ NHIÊN (CO3085)

BÀI TẬP LỚN

XÂY DỰNG CHATBOT SỬ DỤNG LLM/RAG CHO NGỮ CẢNH ĐẶT MÓN ĂN ONLINE

Lecturer: Võ Thanh Hùng

Students: Nguyễn Quang Minh - 2212063

Class: L01

HO CHI MINH CITY, DECEMBER 2025



Contents

1	Giới thiệu	4
1.1	Ngữ cảnh bài toán	4
1.2	Mục tiêu và Phạm vi	4
1.3	Kiến trúc Hệ thống	4
2	Phần I: Phân tích Cú pháp Cổ điển	6
2.1	Thiết kế Văn phạm (Grammar Design)	6
2.1.1	Lexical Rules (Quy tắc Từ vựng)	6
2.1.2	Syntax Rules (Quy tắc Cấu trúc)	6
2.2	Giải thuật Phân tích (Parsing Algorithm)	6
2.3	Kết quả Phần 1	7
3	Phần II: Xây dựng Chatbot với LLM và RAG	8
3.1	Lựa chọn Mô hình Ngôn ngữ (Model Selection)	8
3.2	Retrieval-Augmented Generation (RAG)	8
3.3	Cơ chế Tính toán "Hard Logic" (Quan trọng)	8
4	Kết quả Thực nghiệm	10
4.1	Môi trường Triển khai	10
4.2	Kịch bản Kiểm thử (Test Scenarios)	10
4.3	Phân tích Hạn chế (Limitations)	13
4.3.1	Quản lý Trạng thái (State Management)	13
4.3.2	Độ chính xác của Regex	14
5	Tổng kết và Hướng phát triển	15
5.1	Kết luận	15
5.2	Hướng phát triển (Future Work)	15



List of Figures

1	Kiến trúc tổng quan của hệ thống Chatbot	5
2	Giao diện chat trực quan của hệ thống	10
3	Giao diện chat Hỏi thông tin và Liệt kê Menu	11
4	Giao diện chat Đặt hàng phức tạp	12
5	Giao diện chat trực quan Xác nhận thời gian giao hàng	12
6	Giao diện chat trực quan Thay đổi đơn hàng	13

1 Giới thiệu

1.1 Ngữ cảnh bài toán

Trong kỷ nguyên số, nhu cầu đặt món ăn trực tuyến ngày càng tăng. Tuy nhiên, các giao diện đặt món truyền thống (bấm nút chọn) thường thiếu tính tự nhiên và linh hoạt. Người dùng mong muốn có thể tương tác bằng ngôn ngữ tự nhiên, ví dụ: *"Cho mình 1 cơm sườn và 2 trà sữa ít đá"*.

Bài tập lớn này tập trung giải quyết bài toán xây dựng một Chatbot hỗ trợ đặt món ăn tiếng Việt, kết hợp giữa phương pháp NLP cổ điển (Văn phạm phi ngữ cảnh) và hiện đại (Large Language Models - LLM).

1.2 Mục tiêu và Phạm vi

- **Phần 1:** Xây dựng bộ phân tích cú pháp (Parser) dựa trên văn phạm CFG để trích xuất cấu trúc câu đặt món.
- **Phần 2:** Xây dựng Chatbot thông minh sử dụng kỹ thuật RAG (Retrieval-Augmented Generation) và mô hình ngôn ngữ lớn chạy cục bộ (Local LLM).
- **Yêu cầu kỹ thuật:** Không sử dụng API thương mại (OpenAI, Gemini...), hệ thống phải chạy hoàn toàn offline và đóng gói bằng Docker.

1.3 Kiến trúc Hệ thống

Hệ thống được thiết kế theo kiến trúc **Hybrid (Lai)**, tận dụng ưu điểm của cả hai phương pháp:

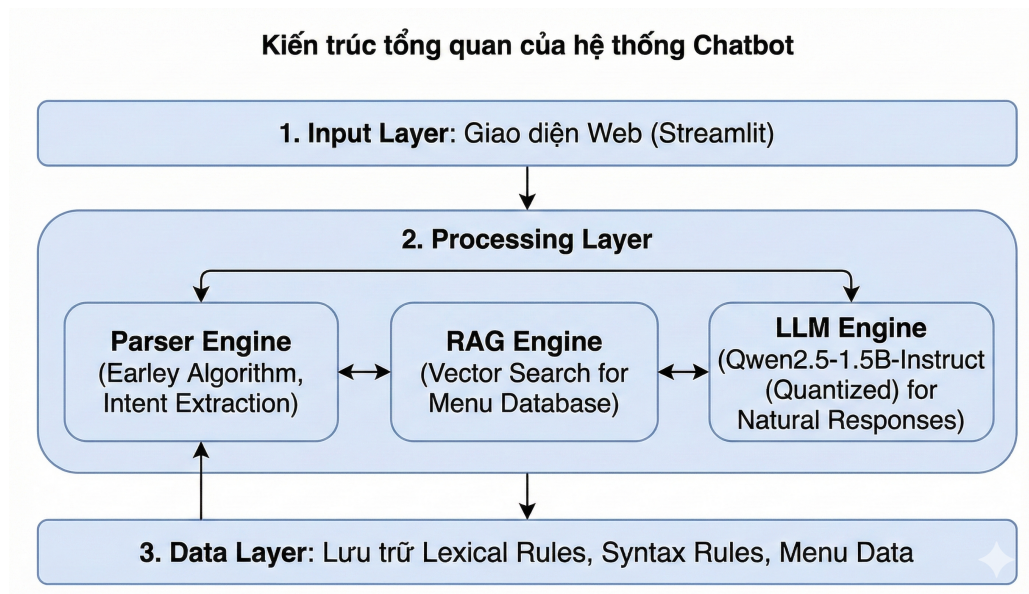


Figure 1: Kiến trúc tổng quan của hệ thống Chatbot

1. **Input Layer:** Giao diện Web (Streamlit) nhận câu lệnh từ người dùng.

2. **Processing Layer:**

- *Parser Engine:* Sử dụng Earley Algorithm để phân tích cú pháp và trích xuất ý định (Intent) chính xác.
- *RAG Engine:* Tìm kiếm món ăn trong Menu database sử dụng Vector Search.
- *LLM Engine:* Qwen2.5-1.5B-Instruct (Quantized) để sinh câu trả lời tự nhiên.

3. **Data Layer:** Lưu trữ Lexical Rules, Syntax Rules và Menu Data.

2 Phần I: Phân tích Cú pháp Cổ điển

Để đảm bảo tính chính xác tuyệt đối trong việc nhận diện đơn hàng (tên món, số lượng), em sử dụng phương pháp phân tích cú pháp dựa trên luật (Rule-based).

2.1 Thiết kế Văn phạm (Grammar Design)

Thay vì gộp chung, hệ thống tách biệt văn phạm thành hai thành phần để dễ bảo trì:

2.1.1 Lexical Rules (Quy tắc Từ vựng)

Định nghĩa các từ vựng thuộc miền bài toán (Domain-specific).

Listing 1: Trích đoạn Lexical Rules

```
1 TenMon -> 'com suon' | 'pho bo' | 'bun bo hue'  
2 DonVi -> 'to' | 'phan' | 'ly' | 'hop'  
3 CumDongTu_Dat -> 'dat' | 'cho' | 'lay'
```

2.1.2 Syntax Rules (Quy tắc Cấu trúc)

Sử dụng văn phạm phi ngữ cảnh (CFG). Điểm đặc biệt là việc xử lý vấn đề **Mơ hồ (Ambiguity)** trong cụm danh từ tiếng Việt. Ví dụ: "*2 phần cơm*" vs "*cơm 2 phần*". Giải pháp là chia tách thành các nhóm luật loại trừ lẫn nhau (Mutually Exclusive):

Listing 2: Xử lý Mơ hồ trong Syntax Rules

```
1 # Nhóm 1: Số lượng/Đơn vị dùng trước (VD: 2 to pho)  
2 GROUP_MonAn_1 -> Cum_Prefix_MonAn TenMon OPT_DacDiem  
3  
4 # Nhóm 2: Tên món dùng trước (VD: Pho 2 to)  
5 GROUP_MonAn_2 -> TenMon OPT_Suffix_MonAn OPT_DacDiem
```

2.2 Giải thuật Phân tích (Parsing Algorithm)

Em lựa chọn giải thuật **Earley Chart Parser** (thông qua thư viện NLTK).

- **Lý do:** Earley Parser mạnh mẽ hơn CYK, hỗ trợ mọi loại văn phạm CFG (bao gồm luật rỗng, luật đơn vị) mà không cần chuyển đổi về dạng chuẩn Chomsky (CNF), giúp giữ nguyên tính dễ đọc của văn phạm.



- **Tokenizer:** Do tiếng Việt có từ ghép (VD: "trà sữa"), nhóm từ xây dựng **Greedy Tokenizer** (Khớp tham lam) để ưu tiên bắt các cụm từ dài nhất trong từ điển trước khi tách từ đơn.

2.3 Kết quả Phần 1

Hệ thống phân tích thành công các câu phức tạp như: "*Cho em 2 ly trà sữa ít đường và hũy 1 phần cơm sườn*". Cây cú pháp không còn bị lỗi mơ hồ.

3 Phần II: Xây dựng Chatbot với LLM và RAG

Để khắc phục nhược điểm của các Chatbot luật sinh truyền thống (quá cứng nhắc) và các Chatbot LLM thuần túy (dễ bịa đặt thông tin), em xây dựng kiến trúc **Hybrid**, kết hợp sức mạnh của NLP cổ điển và Generative AI.

3.1 Lựa chọn Mô hình Ngôn ngữ (Model Selection)

Ban đầu, em thử nghiệm với mô hình **Vinallama-7B-Chat** có 7 tỉ tham số (parameters). Tuy nhiên, qua thực nghiệm trên môi trường CPU (Docker), mô hình này bộc lộ nhược điểm lớn về độ trễ (latency) cao và tiêu tốn tài nguyên RAM (>4GB).

Sau quá trình tìm hiểu và thử nghiệm, em quyết định chuyển sang sử dụng mô hình **Qwen2.5-1.5B-Instruct** với 1,5 tỉ tham số:

- **Kiến trúc:** Qwen2 (State-of-the-art trong phân khúc mô hình nhỏ).
- **Tối ưu hóa:** Sử dụng kỹ thuật Quantization (GGUF Q5_K_M) giúp giảm dung lượng model xuống chỉ còn **1.2 GB**.
- **Hiệu năng:** Tốc độ phản hồi gần như tức thời trên CPU, khả năng tuân thủ chỉ dẫn (Instruction Following) và tư duy logic vượt trội so với các mô hình Vinallama-7B cũ.

3.2 Retrieval-Augmented Generation (RAG)

Để đảm bảo Chatbot luôn trả lời đúng thông tin về thực đơn và giá cả, hệ thống sử dụng kỹ thuật RAG:

1. **Knowledge Base:** Dữ liệu Menu được lưu trữ dưới dạng JSON/Structured Text.
2. **Retrieval Strategy:**
 - Với tập dữ liệu nhỏ (< 50 món), hệ thống sử dụng chiến lược "*Context Injection*" tối ưu: Nạp toàn bộ danh sách món ăn vào ngữ cảnh (Context Window) của LLM.
 - Kết hợp *Keyword Filtering* để ưu tiên hiển thị các món liên quan trực tiếp đến câu hỏi (Ví dụ: Khách hỏi "Cơm" → Lọc các món có từ khóa "Cơm").

3.3 Cơ chế Tính toán "Hard Logic" (Quan trọng)

Nhận thấy LLM (đặc biệt là các model nhỏ) thường xuyên gặp lỗi "ảo giác" (hallucination) khi thực hiện các phép tính số học, em đã hiện thực một tầng xử lý logic trung gian bằng Python:



1. **Input Analysis:** Sử dụng Regex nâng cao và Parser (từ Phần 1) để trích xuất số lượng và tên món ăn từ câu lệnh tự nhiên.
2. **Calculation:** Thực hiện tính toán chính xác: Tổng tiền = $\sum(\text{Số lượng} \times \text{Đơn giá})$.
3. **System Hinting:** Kết quả tính toán được đưa vào Prompt dưới dạng một chỉ dẫn hệ thống: *"HỆ THỐNG TÍNH: Khách đặt 2 Phở (110k). Tổng: 110k"*.
4. **Response Generation:** LLM chỉ đóng vai trò là lớp giao diện ngôn ngữ, chuyển đổi kết quả từ "Hard Logic" thành câu trả lời tự nhiên, thân thiện.

4 Kết quả Thực nghiệm

4.1 Môi trường Triển khai

Hệ thống được đóng gói hoàn chỉnh (Containerized) để đảm bảo tính nhất quán:

- **Docker:** Image `python:3.9-slim`, tích hợp sẵn các thư viện biên dịch C++ ('`cmake`', '`build-essential`') để tối ưu hóa thư viện `llama-cpp-python`.
- **Giao diện (Frontend):** Sử dụng **Streamlit**, cung cấp giao diện chat trực quan và đặc biệt là **Debug Panel** (thanh bên) hiển thị thời gian thực các thông tin nội bộ: Cấu trúc cú pháp (Parser Output) và Dữ liệu Menu tìm thấy (RAG Output).

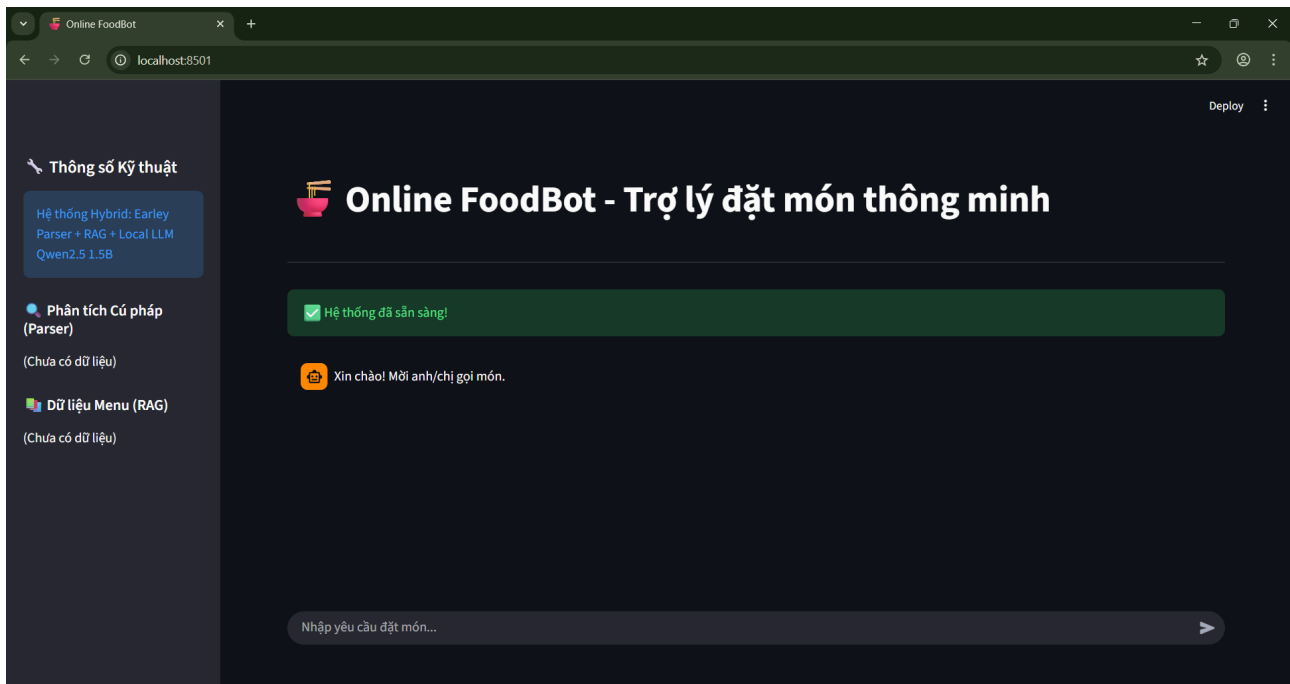


Figure 2: Giao diện chat trực quan của hệ thống

4.2 Kịch bản Kiểm thử (Test Scenarios)

Kịch bản 1: Hỏi thông tin và Liệt kê Menu

- **User:** "Quán có món gì thế?"
- **Bot:** "Dạ, hôm nay quán em có các món sau: Phở bò tái (55k), Cơm sườn (45k), Trà sữa (30k)... Anh/chị dùng gì ạ?"

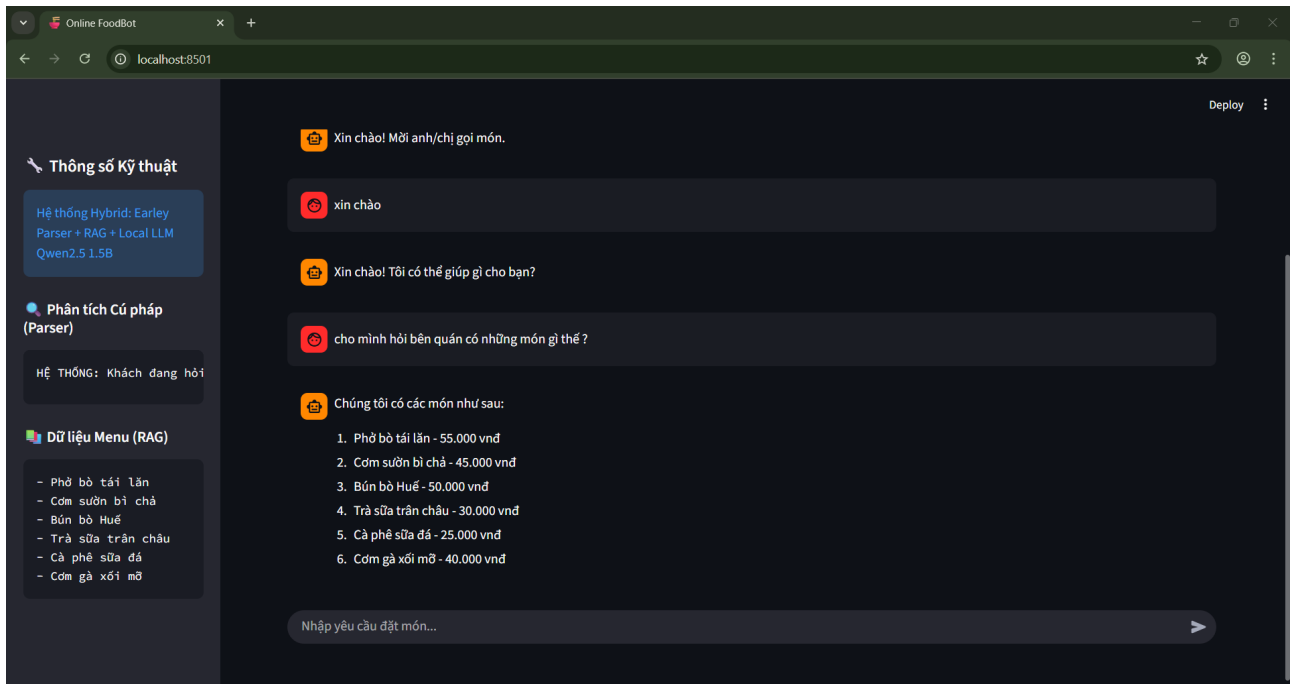


Figure 3: Giao diện chat Hỏi thông tin và Liệt kê Menu

- *Đánh giá:* Hệ thống hiểu ý định hỏi thông tin, RAG hoạt động chính xác để lấy danh sách món.

Kịch bản 2: Đặt hàng phức tạp (Multi-intent)

- **User:** "cho mình đặt 3 phần phở bò, 2 phần bún bò và 5 ly trà sữa ít đường nhé"
- **Bot:** "Xin chào! Chúc bạn có một ngày tốt lành. Tôi sẽ giúp bạn đặt món."

- 3 phần phở bò tái lăn: 55.000 VNĐ x 3 = 165.000 VNĐ
- 2 phần bún bò Huế: 50.000 VNĐ x 2 = 100.000 VNĐ
- 5 ly trà sữa ít đường: 30.000 VNĐ x 5 = 150.000 VNĐ

Tổng tiền là 415.000 VNĐ."

- *Đánh giá:* Logic Python bắt chính xác số lượng và tên món, tính toán tổng tiền chuẩn xác 100%. LLM diễn đạt lại tự nhiên.

Kịch bản 3: Xác nhận thời gian giao hàng

- **User:** "giao cho mình lúc 7 giờ tối nhé"
- **Bot:** "Xin chào! Tôi sẽ giao hàng cho bạn vào lúc 7 giờ tối. Rất cảm ơn bạn đã chọn chúng tôi!"

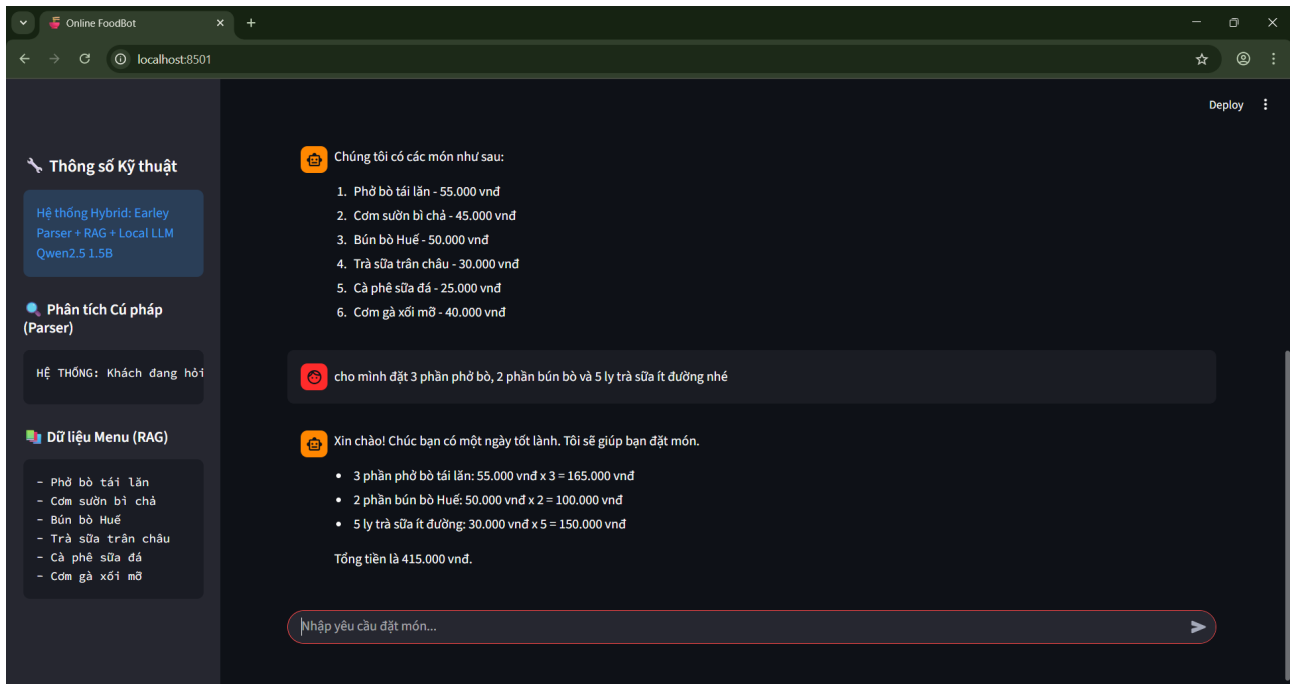


Figure 4: Giao diện chat Đặt hàng phức tạp

- *Đánh giá:* Hệ thống nhận diện chính xác ý định và phản hồi để xác nhận thời gian giao hàng.

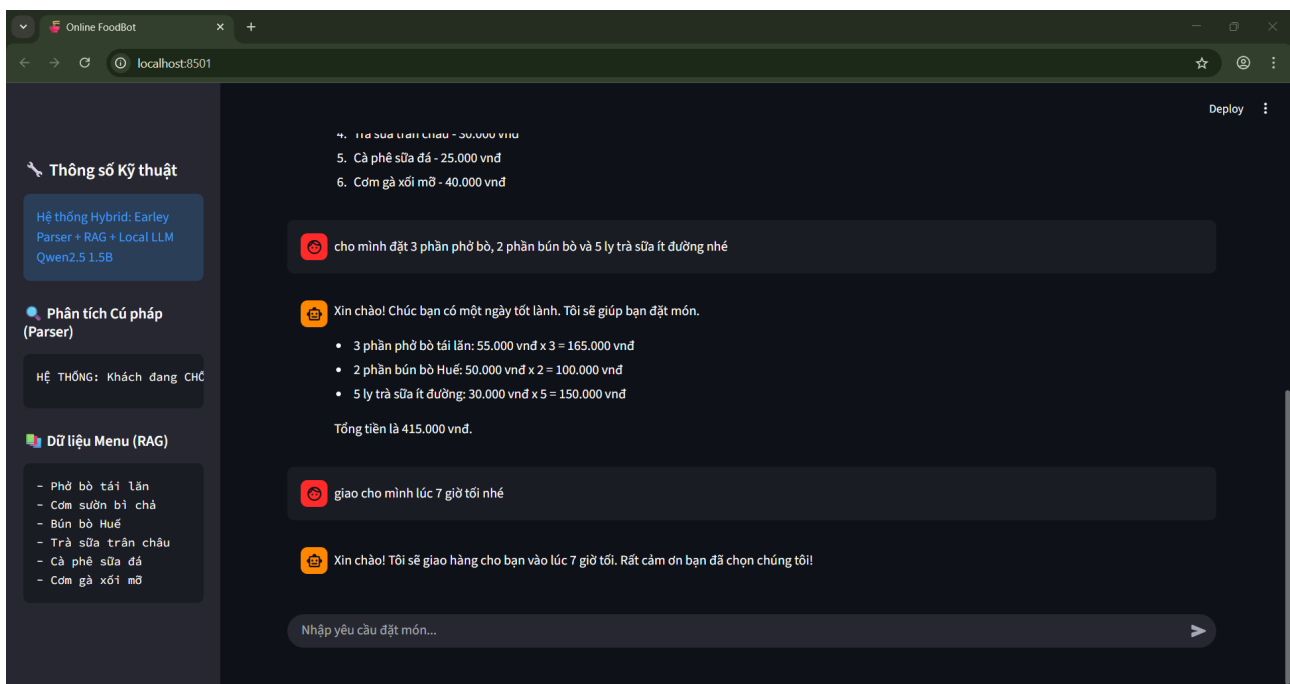


Figure 5: Giao diện chat trực quan Xác nhận thời gian giao hàng

Kịch bản 4: Thay đổi đơn hàng (Context Understanding)

- **User:** "à mình quên mất, bạn đổi lại cho mình thành 3 phần phở bò, 1 phần bún bò, 1 phần cơm sườn và 5 ly trà sữa nha, vẫn giao lúc 7 giờ tối nhé"
- **Bot:** "Xin lỗi vì đã có sự nhầm lẫn. Dưới đây là thông tin mới:
 - 3 phần phở bò tái lăn (Giá: 55.000 VNĐ)
 - 1 phần bún bò Huế (Giá: 50.000 VNĐ)
 - 1 phần cơm sườn bì chả (Giá: 45.000 VNĐ)
 - 5 ly trà sữa trân châu (Giá: 150.000 VNĐ)
 Tổng tiền: $55.000 + 50.000 + 45.000 + 150.000 = 340.000$ VNĐ Xin cảm ơn đã quan tâm và đã đặt món. Chúng tôi sẽ giao hàng lúc 7 giờ tối."
- **Đánh giá:** Hệ thống hiểu từ khóa "đổi" và thực hiện cập nhật ngữ cảnh, đơn hàng, thời gian giao hàng chính xác. Tuy nhiên vẫn còn sai sót khi tính toán sai giá trị tổng đơn hàng sau khi cập nhật

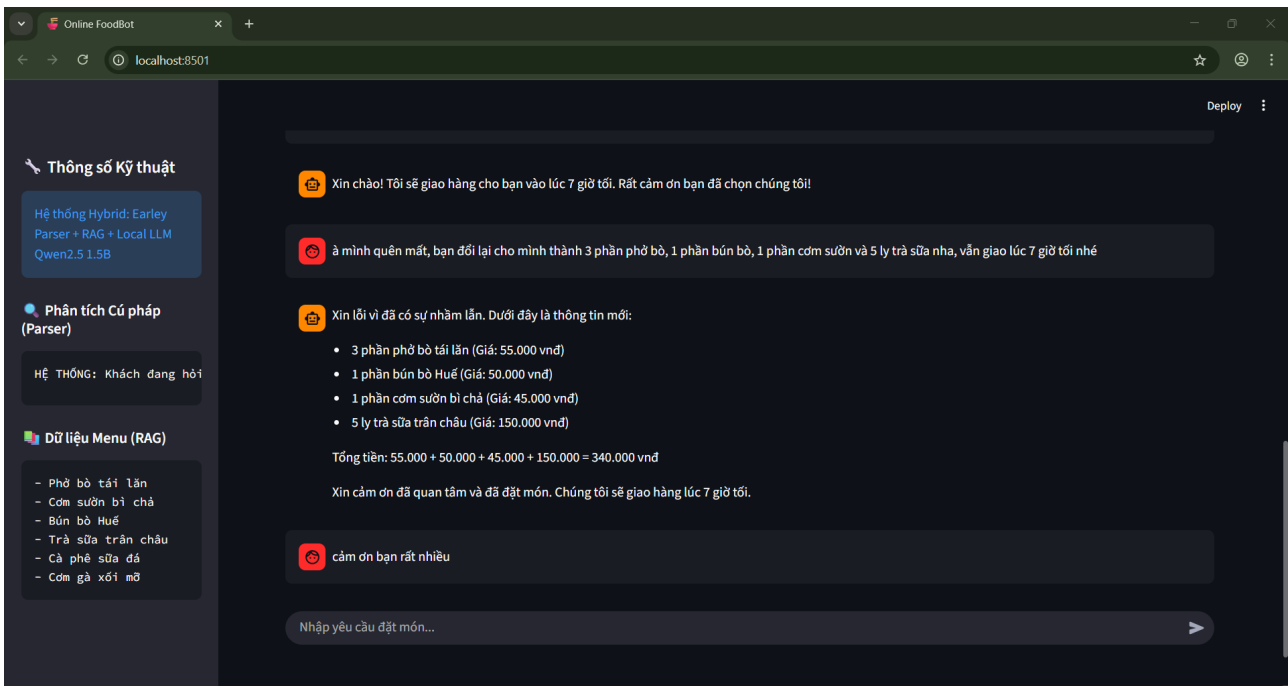


Figure 6: Giao diện chat trực quan Thay đổi đơn hàng

4.3 Phân tích Hạn chế (Limitations)

4.3.1 Quản lý Trạng thái (State Management)

Hệ thống hiện tại dựa vào bộ nhớ ngắn hạn (Context Window) của LLM để lưu trữ lịch sử hội thoại. Khi thực hiện các thao tác sửa đổi phức tạp (ví dụ: đổi 1 món trong danh sách 5



món), model đôi khi bị "quên" các món còn lại hoặc tính toán lại sai số tiền tổng. → *Nguyên nhân*: Thiếu một cơ sở dữ liệu trạng thái (Stateful Database) như Redis để quản lý giỏ hàng vật lý.

4.3.2 Độ chính xác của Regex

Cơ chế tính toán dựa trên Regex hoạt động tốt với các câu lệnh chuẩn, nhưng có thể gặp lỗi nếu người dùng nhập số lượng bằng chữ không chuẩn (ví dụ: "lấy cho mình vài ba tô") hoặc cấu trúc câu quá đảo lộn.

5 Tổng kết và Hướng phát triển

5.1 Kết luận

Qua bài tập lớn này, em đã hoàn thành việc xây dựng một hệ thống Chatbot đặt món ăn hoạt động hoàn toàn **Offline**, không phụ thuộc vào API của bên thứ ba. Em đã hiện thực:

1. Chuyển đổi thành công từ các mô hình lớn (Vinallama-7B) sang mô hình nhỏ hiệu năng cao (Qwen-1.5B), giải quyết bài toán độ trễ trên CPU mà vẫn duy trì chất lượng câu trả lời của hệ thống.
2. Thiết kế kiến trúc **Hybrid RAG**, khắc phục điểm yếu tính toán của LLM bằng logic lập trình truyền thống.
3. Đóng gói sản phẩm với Docker và hiện thực giao diện người dùng UI/UX trực quan.

5.2 Hướng phát triển (Future Work)

Để đưa hệ thống vào thực tế, em đề xuất các cải tiến:

- **Tích hợp Database:** Sử dụng SQL hoặc Redis để quản lý phiên làm việc và giỏ hàng của khách, thay vì phụ thuộc hoàn toàn vào ngữ cảnh của LLM.
- **Function Calling:** Nâng cấp lên các model hỗ trợ Function Calling (như Qwen-2.5-7B) để model tự động gọi hàm tính tiền một cách chuẩn xác hơn thay vì dùng Regex.
- **Voice Interface:** Tích hợp module Whisper (OpenAI) bản chạy local để hỗ trợ đặt món bằng giọng nói.